Aggressive vs. Conservative TTG

Latest accepted results: Access coverage and unnecessary full-node fetches

JDrive

Aggressive: at least 832
unnecessary full-node fetches.
Conservative: 0 confirmed
in the audited classes.

LittleX

357 prefetched and used
in both policies.
Access-only reads are
covered by Conservative.

RecipePantry

Conservative: 118 / 164 covered
71.95% demand coverage
0 unused prefetches
Semantic waste: not audited

GIL-off threads • n = 3 per reported configuration • native results preserved

RecipePantry Conservative uses the November 11 condition-call repair.
The other configurations retain their November 10 captures; Aggressive was not rerun.

Define “unnecessary” at the payload boundary

A node need not be visited to be useful; an accessed payload can still be unnecessary.

Necessary payload

Fields are needed for the result or later computation. An ordinary helper read counts, even without Visit.

Filter-only payload

Full node fetched only to reject it using a predicate. SQL can evaluate the filter without returning the node.

Unused prefetch

Prefetched payload never read by foreground code. This is one measurable class of unnecessary fetch.

Observed coverage = demanded IDs returned by prefetch / all demanded IDs

Counts are unique within each HTTP request / Session, then summed over the workflow.
The denominator includes root reads and memory hits. This is not a database cache-hit rate.

Confirmed unnecessary = unused + disjoint, audited filter-only payloads (a lower bound).
Zero unused does not establish zero semantically unnecessary reads.

Fixed workloads and explicit measurement provenance

The summary combines accepted captures; it is not a simultaneous new three-app rerun.

App	Business workflow	Capture used
JDrive	List / visible tree / trash / create folder	Nov 10: both policies
LittleX	Feed / likes / follow / profile / tweet	Nov 10: both policies
RecipePantry	Plan / buy / cook / boards / shopping	Nov 10 Agg.; Nov 11 Cons.

Same RecipePantry scenarios and business results

The repair changes Access extraction; native control flow, responses and persisted state agree.

Thread executor, GIL disabled, 1 worker, limit 2048. Each reported configuration has n=3.
The Nov 11 validation ran 8 fresh workflows: 2 excluded warmups + 6 measurements.
Only the repaired Conservative results are presented here.

Aggressive vs. Conservative: summary

Hit rate means observed demand coverage; unnecessary means avoidable full-node payload.

App	Aggressive hit rate	Conservative hit rate	Aggressive unnecessary fetches	Conservative unnecessary fetches
JDrive	91.40%	92.01%	At least 832	0 confirmed
LittleX	92.97%	92.97%	0 confirmed	0 confirmed
RecipePantry	67.07%	71.95%	At least 3	Not audited

RecipePantry Conservative: 118 / 164 covered; 0 unused, semantic waste not audited.

RecipePantry Aggressive is the earlier 112 / 167 capture, not a rerun on the repaired version.
“0 confirmed” means no examples in the audited classes, not proof of zero unnecessary reads.

Request-local unique IDs summed per workflow; all three repetitions agree.

Payload counts: use is not the same as necessity

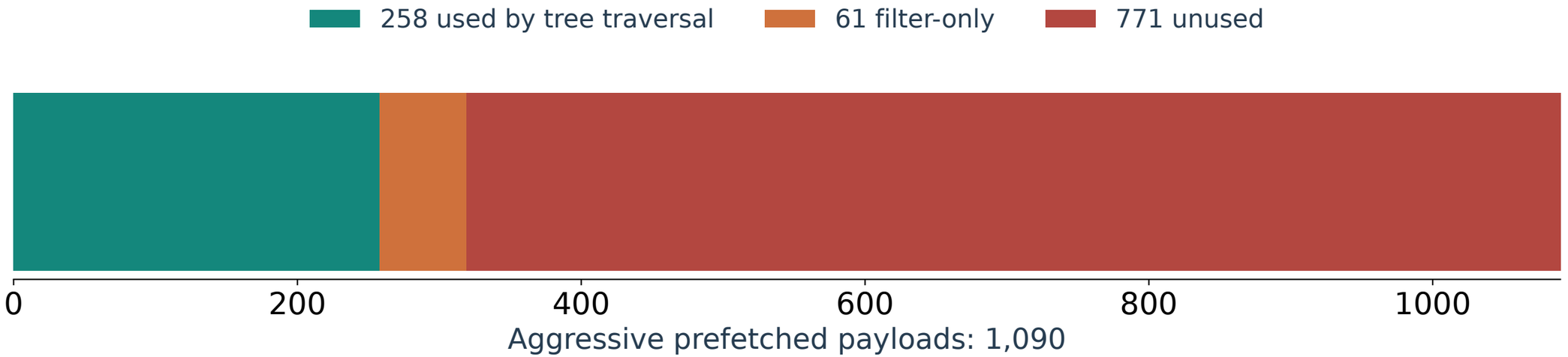
Latest accepted counts per configuration; all three repetitions agree

App / policy	Prefetched	Used	Unused	Filter-only	Unnecessary lower bound
JDrive / Agg.	1090	319	771	61	832 min.
JDrive / Cons.	265	265	0	0	0 found
LittleX / Agg.	357	357	0	0	0 found
LittleX / Cons.	357	357	0	0	0 found
RecipePantry / Agg.	112	112	0	3	3 min.
RecipePantry / Cons.	118	118	0	Not audited	Not audited

RecipePantry Conservative: all 118 prefetched objects were subsequently demanded. Ownership membership can still be implemented with IDs instead of complete payloads; that semantic optimization has not been quantified in this capture.

JDrive: unused and filter-only payloads

The trashed == False predicate avoids full-node fetches for rejected candidates.



Aggressive: at least 832

771 unused + 61 rejected trashed candidates with avoidable full payloads.

Conservative: 265 used

258 tree nodes + 7 ListFolder Accesses.
0 unnecessary examples in audited classes.

November 10 capture retained. Matched business outputs and final states; no new JDrive run.

LittleX: Access-only demand is real demand

357 prefetched and used in both policies; 92.97% observed coverage



Conservative payload use	Count
Prefetched and entered	287
Useful without Visit	70
Total used prefetch	357

A helper read is a valid demand even if the walker never visits the object.
November 10 accepted capture retained; no new LittleX run.

RecipePantry: repaired Conservative results

Condition calls and ordinary calls now share Access extraction; GIL-off threads, n=3.

Measure	Current result
Observed foreground demand	164
Prefetched / subsequently used	118 / 118
Observed demand coverage	118 / 164 = 71.95%
Unused prefetch / uncovered demand	0 / 46
Data / control / total SQL	213 / 166 / 379

All counts agree across the three measured workflows. Business outputs and persisted state pass.
Uncovered demand remains visible; zero unused is not a semantic-minimality claim.
These precision captures alone do not establish a latency speedup.

One Access analysis for helpers and conditions

Prefetch supported reads when their real dependencies are available; keep control flow native.

1 Bind the object

meal = jobj(self.meal_id)
Native lookup materializes the actual Meal object.

2 Prefetch helper Access

Before _owns(meal, Meal):
check the actual safe type;
prefetch [root-->] demand.

3 Follow native control

Execute _owns once.
After ownership / status:
prefetch _requirements.

```
isinstance(item, kind) and jid(item) in [jid(n) for n in [root-->]]
```

Short-circuit guards remain part of the demand

A false AND-left or true OR-left prevents right-side prefetch; opaque predicates run natively.

Unknown predicate results do not discard a supported helper Access prefix.
Hooks can resume at supported statement boundaries after actual local bindings become available.

SQL counts are a separate outcome

Latest accepted data / control counts; request-local object coverage is not SQL hit rate.

App	Aggressive data / control	Conservative data / control
JDrive	208 / 52	53 / 54
LittleX	74 / 70	73 / 71
RecipePantry	189 / 150	213 / 166

RecipePantry covers 118 demands with 213 data SQL calls

Access extraction supplies demand information; it does not fuse every path or payload query.

RecipePantry Aggressive is an earlier capture; Conservative is the repaired November 11 version.
A Session demand can be satisfied from memory. A higher coverage ratio does not by itself
prove fewer SQL calls, less semantic waste, or lower latency.

What to improve next

Optimize the demand representation as well as the number of demands covered.

1 Separate identity, predicate and full-payload needs

Ownership membership uses IDs; avoid fetching complete neighbors when IDs are sufficient.

2 Cover the remaining actual demand

RecipePantry still has 46 uncovered request-local demand IDs. Preserve control-flow guards.

3 Fuse compatible accesses and measure overhead

Track path / payload / control SQL separately; re-test request wall time with matched instrumentation.

4 Re-audit semantic necessity

Report coverage, unused prefetch and unnecessary full payload separately. Keep failed opportunities visible.

Evidence and scope

Current results are bound to accepted sources, captures and reproducible derivation.

RecipePantry repair: 8 fresh workflows, 6 measured

3 measured per version plus excluded warmups; only repaired results are shown in this deck.

Validation: 61 pure tests + 89 native regression tests

Native outputs / state / restart / GIL-off and all eight owned database cleanup checks passed.

Raw-derived results, with explicit provenance

256 new sealed files checked; all n3 counts match. Original three-app evidence is preserved.

Editable render_v3.py + derive_update.py + copied accepted results / validation evidence.
The new raw archives remain in the measurement workspace; the package retains its original 24 archives.
RecipePantry semantic waste was not re-audited. Aggressive / JDrive / LittleX were not rerun.